Lecture 8
Chapter 14: Indexing in Database

Md. Tariquzzaman
Junior Lecturer, Dept. of Computer Science and Engineering
Islamic University of Technology (IUT)

CSE 4307: Database Management Systems

Chapter Outline

- Indexing: Basic Concept
- Index Types: Primary and Secondary Index
- Index Types: Dense and Sparse Index
- B Tree
- B Tree Insertion
- B Tree Deletion
- B+ Tree

Indexing: Basic Concept

Basic Idea

Index

An **index** is a data structure that helps in speeding up the retrieval of data from the database tables. It is similar to the index at the back of a book that helps in locating the topic quickly.

—Index—

–A–

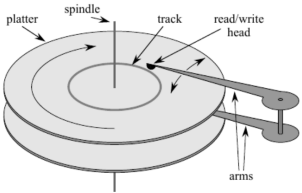
about the author 128, 132, 412
account info 295
active table of contents 34, 120-124, 238-239, 285-286, 354, 366, 370
ACX 465-467
Adobe 506
advertising 434, 439-449
age 312
aggregator 17-18, 322
alignment 68, 101-103, 105-106, 229-230, 261-262, 353-354, 380, 389
Alt codes 39
Amazon Associates 415
Amazon Follow 430, 437, 480
Amazon Giveaway 436-439
Amazon Marketing Services (AMS) 439-449
Android 167-169, 171, 371-373
apostrophe 40, 42-44
app 141-142
Apple 169, 342, 372, 506
automatic renewal 327-329, 341, 343
Automatically Update 73-75, 94, 144
AZK 371

–B–

back matter 124-129
background 47, 93, 181, 184, 192-193, 246, 252-253, 355, 370, 385, 390
bank information 295
Barnes & Noble 506
biography 128, 132, 410
black 47, 93, 184, 192, 252-253, 355, 370, 385, 390
Blackberry 372-373
blank line 27-28, 110, 112-114, 276-277, 284-285, 385
blank page 354, 385-386
block indent 50, 52, 67, 82, 106-107, 234-235
blog 411, 429, 479
Blogger 429
bloggers 327, 430
blurb 300-306, 364, 406, 411-412, 417, 477
blurry 162-164, 172, 175, 193, 246, 387, 389
body text 66, 68, 79-82, 92-94, 115, 233-235

Background: Data Structure on Secondary Storage

- The **primary memory** (or main memory) of a computer system normally consists of silicon memory chips. It is **expensive** and **faster**.
- Most computer systems also have **secondary storage** based on solid-state drives (SSD) or magnetic disk drives. In recent years SSD have become cheaper with lower price.
- Primary memory is **not suitable** to handle database records because of its large volume.
- Because of the moving parts (i.e. platters, arms) in disk drives, it is **slower** than primary memory by a number of factors.
- Hence **indexing** is essential and critical for handling database records in a real-life system.



Simple Indexing Will Not Work

Implementing an index on the student relation by keeping a **sorted list of student IDs** would **not work well on very large databases**. Because:

1. The index would itself be **very big**
2. Even though keeping the index sorted reduces the search time, finding a student can still be rather **time-consuming**
3. **Updating** a sorted list as students are added or removed from the database can be **very expensive**

Hence, **more sophisticated indexing techniques** are used in database systems.

Indexing: Broad Classification

There are two basic kinds of indices:

1. **Ordered indices** — based on a **sorted ordering** of the values.
2. **Hash indices** — based on a **uniform distribution** of values across a range of buckets. The bucket to which a value is assigned is determined by a function, called a **hash function**.

Note

In this lecture, concentration will be on **ordered indexing**.

Indexing Technique: Evaluation Criteria

No Single Best Technique

No one technique is best for all cases. Each technique must be evaluated based on the specific database application.

Each indexing technique must be evaluated on these factors:

1. **Access types** — the types of access supported efficiently (e.g., exact value match or range query)
2. **Access time** — the time to find a particular data item or set of items
3. **Insertion and deletion time** — the time to insert or remove a data item, including the cost of updating the index structure
4. **Space overhead** — the additional space occupied by the index structure

Basic Concept

- Indexing mechanisms are used to **speed up access** to desired data (e.g., an author catalog in a library)
- **Search key** — an attribute or set of attributes used to look up records in a file
- An **index file** consists of records called **index entries** of the form:

search-key	pointer

- Index files are typically **much smaller** than the original file
- Two basic kinds of indices:
  - **Ordered indices** — search keys stored in sorted order
  - **Hash indices** — search keys distributed uniformly across buckets using a hash function

Search Key

Search Criteria

An attribute or set of attributes used to look up records in a file is called a **search key**. Note that this definition of **key differs** from that used in primary key, candidate key, and superkey. Multiple search keys are common in use.

Ordered Index: Types

A file may have several indices on different search keys. An ordered index can be of two types:

1. **Clustering Index** — a type of index which sorts the data rows in the table on their key values. In the database, there is **only one clustered index per table**. A clustered index defines the order in which data is stored. In an RDBMS, usually, the **primary key** allows you to create a clustered index. Also called **Primary Index**.
2. **Nonclustering Indices** — a non-clustered index stores the data at one location and indices at another location. The index contains pointers to the location of that data. A single table can have **many non-clustered indexes**. The index is stored in different places. Called **Secondary Index**.

Index Types: Primary and Secondary Index

Clustering Index

Definition

If the file containing the records is sequentially ordered, a **clustering index** is an index whose search key also defines the sequential order of the file.

Properties:

- Since both index and records are in the **same physical block**, searching is **very fast**
- Because the data can only be sorted in one way, you can have **only one clustered index per table**
- Clustering indices are also called **Primary Indices** (the term primary index may appear to denote an index on a primary key)
- The search key of a clustering index is **often the primary key**, although that is not necessarily the only option
- The **leaf nodes** of a clustered index contain the **actual data rows**

Non-Clustering Index

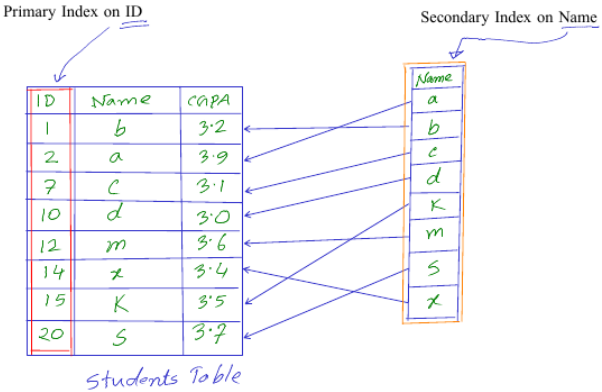
Definition

Indices whose search key specifies an order different from the sequential order of the file are called **nonclustering indices**, or **secondary indices**. In simple word, **a non-clustered index is stored in one place and the actual table data is stored in another**.

Properties:

- Index and records are in **different physical blocks** — searching is **a bit slower**
- Non-clustering indices are also called **Secondary Indices**
- It is possible to have **multiple non-clustered indexes on a single table**

Primary and Secondary Index: Concepts



Index Types: Dense and Sparse Index

Index Entry

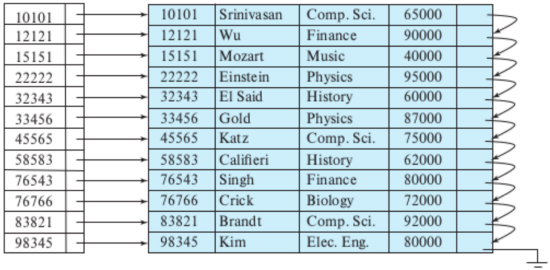
Index Entry

An index entry, or **index record**, consists of a **search-key value** and **pointers** to one or more records with that value as their search-key value. The pointer to a record consists of the identifier of a disk block and an **offset** within the disk block to identify the record within the block.

Dense Index

Dense Index: Definition

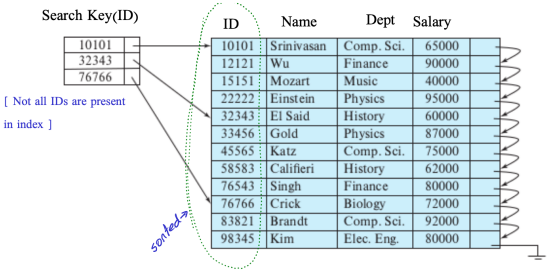
In a **dense index**, **an index entry appears for every search-key value** in the file. In a dense clustering index, the index record contains the search-key value and a pointer to the **first data record** with that search-key value.



Sparse Index

Sparse Index: Definition

In a **sparse index**, an index entry appears for **only some of the search-key values**. Sparse indices can be used only if the relation is stored in **sorted order** of the search key; that is, if the index is a clustering index.



Dense and Sparse Index: Pros and Cons

Dense Index:

- It is generally **faster to locate a record** if we have a dense index rather than a sparse index

Sparse Index:

- Sparse indices have advantages over dense indices in that they **require less space** and they impose **less maintenance overhead** for insertions and deletions

Access Time and Space Overhead: A Trade-off

- There is a **trade-off** that the system designer must make between **access time** and **space overhead**
- Trade-off depends on the specific application
- The **dominant** cost in processing a database request is the time that it takes to **bring a block from disk into main memory**
- Once we have brought in the block, the time to scan the entire block is **negligible**

Multilevel Indices: Motivation

- If an index is **small enough to be kept entirely in main memory**, the search time to find an entry is low
- However, for a larger index, the search for an entry then requires **several disk-block reads**
- Binary search can be used on the index file to locate an entry, but the search still has a large cost. If the index would occupy b blocks, binary search requires as many as $\lceil \log_2 b \rceil$ blocks to be read. Even this is **quite high for a very large database**
- To deal with this problem, we treat the index just as we would treat any other sequential file, and we construct a **sparse outer index** on the original index, which we now call the **inner index**

Multilevel Indices: A Simple Analogy

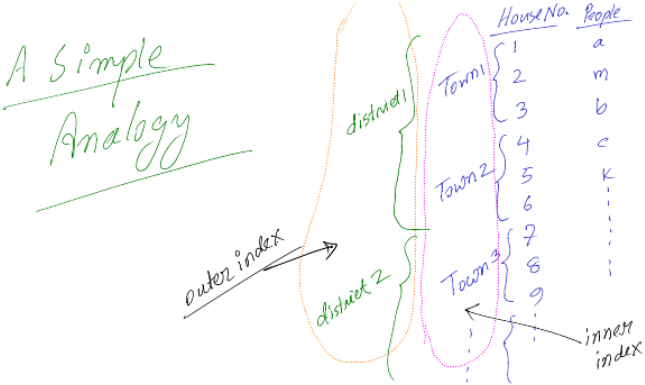


Figure: Multi-level index: Analogy

Multilevel Indices: Structure

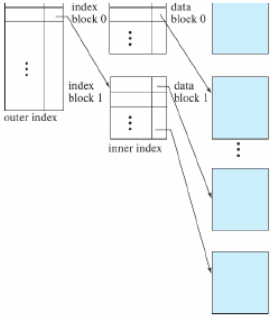


Figure: Multi-level index: Basic Idea

Binary Search vs. Multi-Level Search

Binary Search vs Multi-Level Search

In binary search tree the number of read operations to search one item is bounded by $\log_2 n$ where n is the number of items in the storage. Actually binary search tree is just a special case with $m = 2$, the higher the value of m the lower the height of the tree, the faster the search. In database the term m which is eventually the number of maximum children per node is often termed as **Fan Out**.

Example: Suppose a table has 1,000,000 tuples, each block (of size 4K) contains 100 records. Hence total number of blocks needed is:

$$\frac{1{,}000{,}000}{100} = 10{,}000 \text{ blocks}$$

Binary Search vs. Multi-Level Search (cont.)

Key Insight

Higher fan-out (m) dramatically **reduces the number of disk accesses** compared to binary search ($m = 2$), especially for large databases.

Value of m	No. of reads needed
$m = 2$	$\lceil \log_2 10000 \rceil = \lceil 13.44 \rceil = 14$
$m = 5$	$\lceil \log_5 10000 \rceil = \lceil 5.7 \rceil = 6$
$m = 10$	$\lceil \log_{10} 10000 \rceil = 4$
$m = 50$	$\lceil \log_{50} 10000 \rceil = \lceil 2.35 \rceil = 3$

B and B+ Tree

B/B+ Tree: Concepts to Recap

To cover this topic we need to recap a few concepts:

- How data is stored on disk (**blocks vs. records**)
- **m-way search trees**
- **B Trees**
- **B+ Trees**

Blocks vs. Records

In a database, data is **not** read one record at a time. It is read in **blocks (or pages)**. A single block typically contains many records. So, the smallest unit of storage access is blocks.

The size of each block in PostgreSQL is 8KB while it also supports 4KB in Linux systems.

Goal of Indexing

The goal of indexing is to **reduce the number of block reads** to fetch records upon user request.

m-way Search Tree

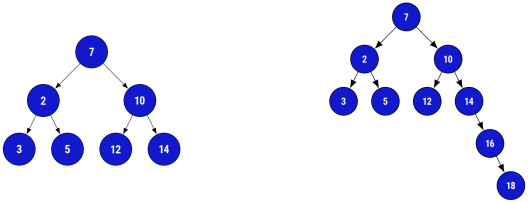


Figure: BST

- In BST we have **1 key** and **2 children**
- In m-way Search Tree it may have **more than** 1 key and more than 2 children

Properties:

- **Node Capacity** — Each node can contain at most $(m-1)$ **keys** and m **children**
- **Key Arrangement** — The keys within a single node are always stored in **sorted order** (e.g., $K_1 < K_2 < K_3 \ldots$)

m-way Search Tree: Modification for Database

Why Modify It for Databases?

In a regular m-way tree used for algorithms, each node only stores the key value. But in a database index, we need to actually **find the full record** in the table!

The Problem:

- Index Node: [Key: 101]
- Where is the actual student record with ID 101?

Database Table Structure: The index is separate from the actual data table:

INDEX FILE

Key	Pointer
100	$\rightarrow$ Block 5
200	$\rightarrow$ Block 12
300	$\rightarrow$ Block 20

DATA FILE (Table)

Block 5: [98, 99, **100**, 101, 102]

Block 12: [198, 199, **200**, 201]

Block 20: [298, 299, **300**, 301]

The Modification: Record Pointer (R_p)

Each key in the index needs a pointer to the actual data record on disk.

Modified Node Structure (for $m = 4$):

- P_0 | K_1 | P_1 | K_2 | P_2 | K_3 | P_3
- Where P = Child Pointer, K = Key
- Plus: R_{p1}, R_{p2}, R_{p3} = Record Pointers for each key

Complete Structure:

Component	Purpose	Count
Keys ($K_1, K_2, \ldots$)	Search values	$m - 1$
Child Pointers ($P_0, P_1, \ldots$)	Point to child nodes	m
Record Pointers ($R_{p1}, R_{p2}, \ldots$)	Point to actual data records	$m - 1$

Why This Matters (Real Example)

Student Table (1 million records)

- Index Node: [ID: 100 | ID: 200 | ID: 300]
- $R_{p1} \rightarrow$ Disk Block 5, Offset 12 (Student #100's record)
- $R_{p2} \rightarrow$ Disk Block 8, Offset 45 (Student #200's record)
- $R_{p3} \rightarrow$ Disk Block 12, Offset 3 (Student #300's record)

Without Record Pointer

"I found ID 200 in the index... but where is the actual data??"

With Record Pointer

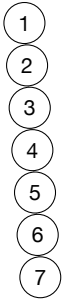
"ID 200 is at Block 8, Offset 45" $\rightarrow$ Direct access to full record

The Balance Problem

Regular m-way trees (like BSTs) can become **unbalanced**:

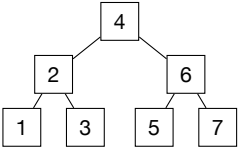
Unbalanced BST (Worst Case)

Insert: 1, 2, 3, 4, 5, 6, 7



Solution: B-Tree (Balanced)

Same data with $m = 3$:



Search time: $O(\log_m n)$

This is basically a **linked list**! Search: $O(n)$

The Balance Problem (cont.)

Key Takeaway

B-Tree = m-way Search Tree + Record Pointers + **Balance Guarantees**

B-Tree: Structure, Rules and Properties

A **B-Tree** has the identical node structure of an m-way Search Tree with additional rules:

1. Node Structure

- $m - 1$ keys
- m child pointers
- $m - 1$ record pointers

2. Balance Rules

- All leaves at **same level**
- $\lceil m/2 \rceil$ children min (non-root)
- **Bottom-up** construction

3. Properties (for $m = 5$)

Children:

- Max: m = 5
- Min: $\lceil 5/2 \rceil$ = 3

Keys:

- Max: $m - 1$ = 4
- Min: $\lceil 5/2 \rceil - 1$ = 2

4. Root Special Case

- Min keys: 1
- Min children: 2

Key Formula

Min children = $\lceil m/2 \rceil$
Min keys = $\lceil m/2 \rceil - 1$
in B/B+ trees, m is the order (maximum no. of child in a node), which means it can have m-1 keys in the parent

B Tree Insertion

B-Tree Insertion: Overview

1. **Search** for the appropriate leaf node:
   - Start at the **root**, compare key with node values
   - Follow **child pointers**: $K_i <$ key $< K_{i+1}$ goes to child P_i
   - Repeat until reaching a **leaf node**
2. **Insert** the key into the leaf node in sorted order.
3. **Check** if the node exceeds capacity:
   - If **node has space**: Insertion complete.
   - If **node is full**: **Split** the node.

B-Tree Insertion: Overview (cont.)

Node Splitting Process

1. Sort all keys (including new one)
2. Select **mid value** as the key to move up
3. Create two new nodes from remaining keys
4. If parent is full, split propagates upward (may create new root)

Interactive B-Tree Visualization

Go to this link to simulate and try B-Tree:

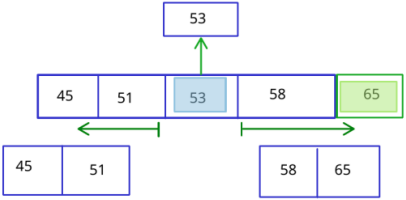
`https://www.cs.usfca.edu/~galles/visualization/BTree.html`

Tip

Use this visualization tool to practice insertion and deletion operations. Set different values of m and see how the tree structure changes!

B-Tree Node Splitting

This is the **fundamental step** in constructing a B-Tree. The key values (including the new one) will be **sorted** and then the **mid value** will be selected to pick it up as the immediate root. Node Splitting occurs when there is no space to hold another item.

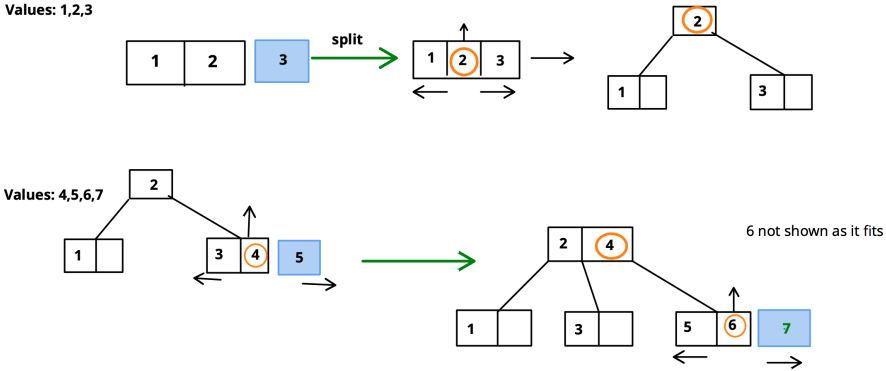


B Tree Construction: Example 1

Consider a B-Tree with $m = 3$ (max 3 children, 2 keys per node).

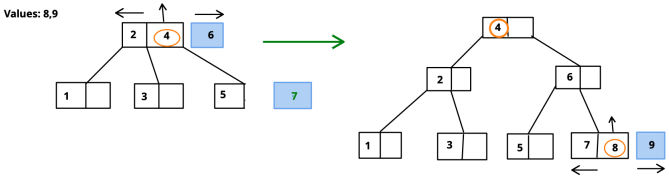
Dataset: **{1,2,3,4,5,6,7,8,9,10}**

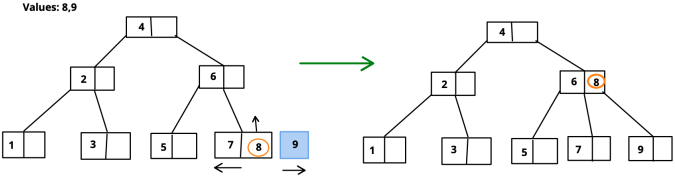
Insert keys: **1, 2, 3, 4, 5, 6, 7**



B Tree Construction: Example 1 (cont.)

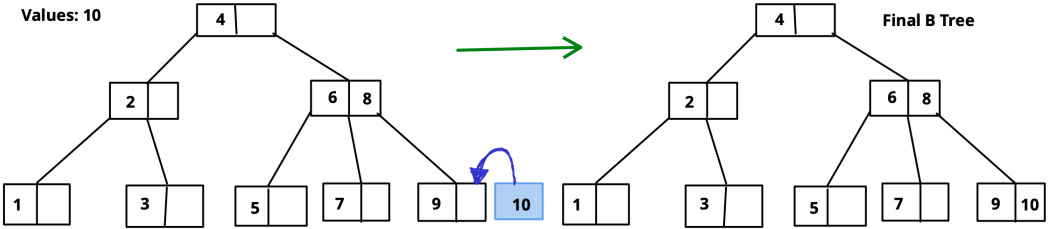
Insert keys: **8, 9**





B Tree Construction: Example 1 (cont.)

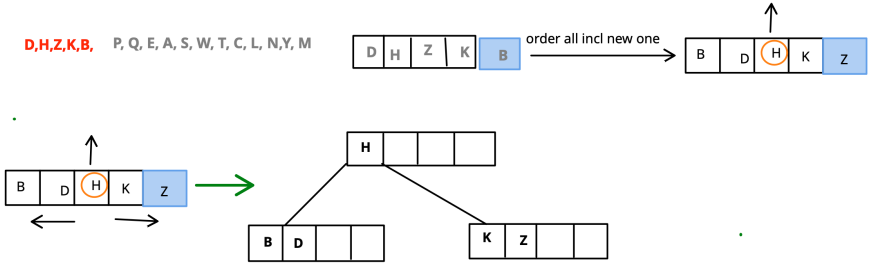
insert keys: **10**



B Tree Construction: Example 2

B-Tree with $m = 5$, Dataset: **D,H,Z,K,B,P,Q,E,A,S,W,T,C,L,N,Y,M**

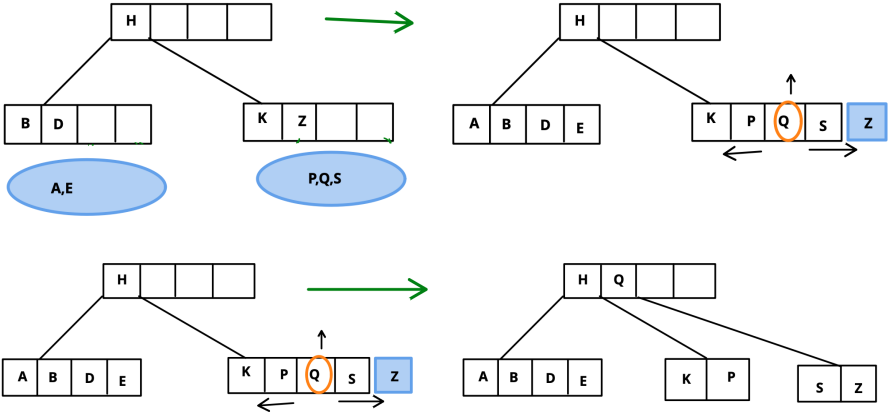
Insert keys: **D,H,Z,K,B**



B Tree Construction: Example 2 (cont.)

Insert keys: **P, Q, E, A, S**

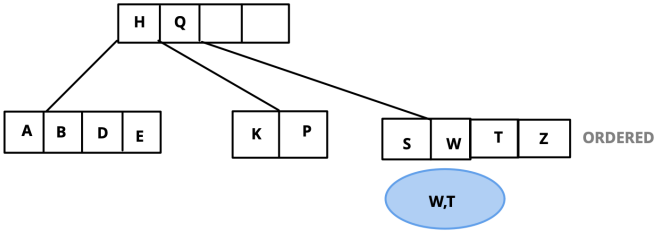
D,H,Z,K,B, **P, Q, E, A, S,** W, T, C, L, N,Y, M



B Tree Construction: Example 2 (cont.)

Insert key: **W, T**

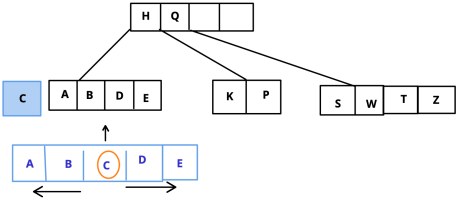
D,H,Z,K,B, P, Q, E, A, S, **W, T,** C, L, N,Y, M (Can be fit in one node)

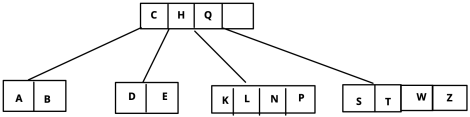


B Tree Construction: Example 2 (cont.)

Insert key: **C**

D,H,Z,K,B, P, Q, E, A, S, W, T, C , L, N,Y, M

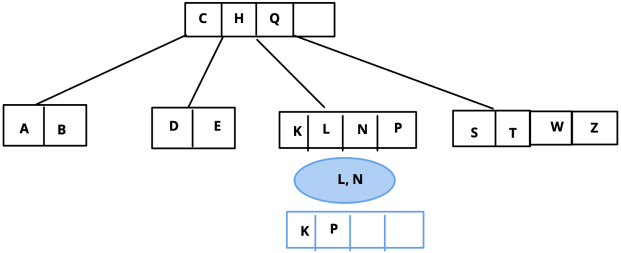




B Tree Construction: Example 2 (cont.)

Insert key: **L,N**

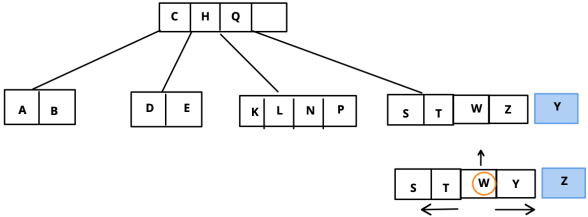
D,H,Z,K,B, P, Q, E, A, S, W, T, C , L, N, Y, M

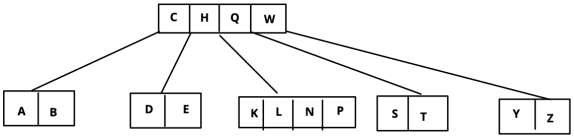


B Tree Construction: Example 2 (cont.)

Insert key: **Y**

D,H,Z,K,B, P, Q, E, A, S, W, T, C , L, N, Y ,M

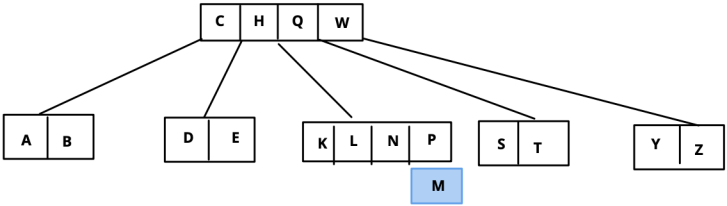




B Tree Construction: Example 2 (cont.)

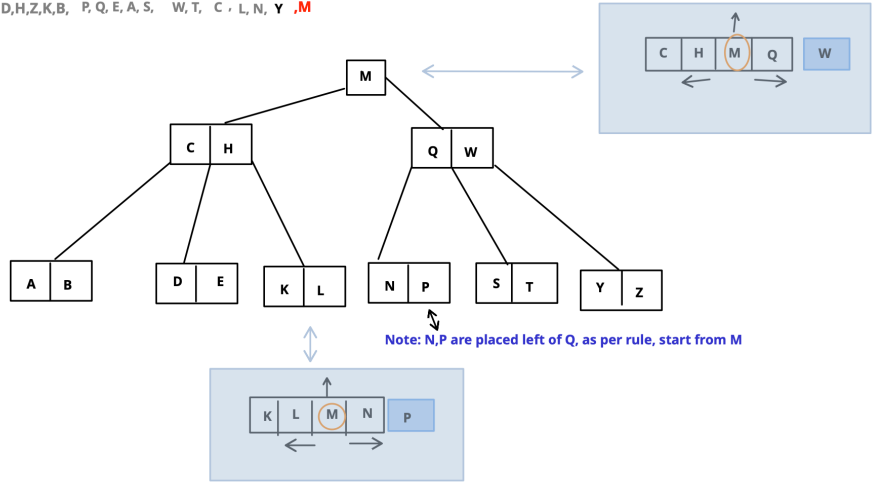
Insert keys: **M**

D,H,Z,K,B, P, Q, E, A, S, W, T, C , L, N, Y ,M



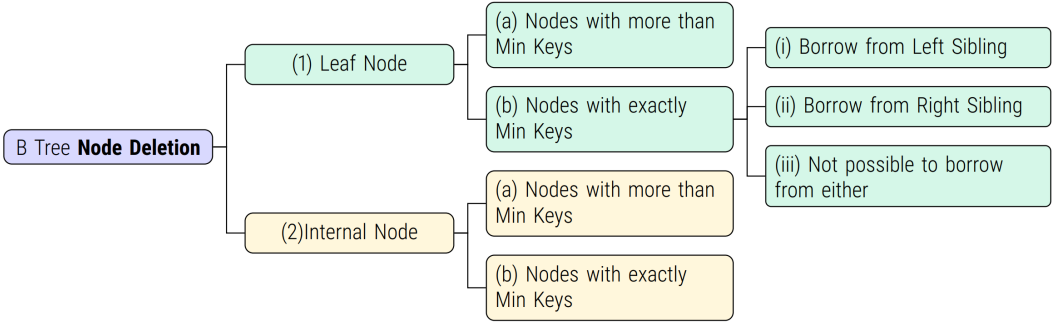
B Tree Construction: Example 2 (cont.)

D,H,Z,K,B, P, Q, E, A, S, W, T, C , L, N, Y ,M



B Tree Deletion

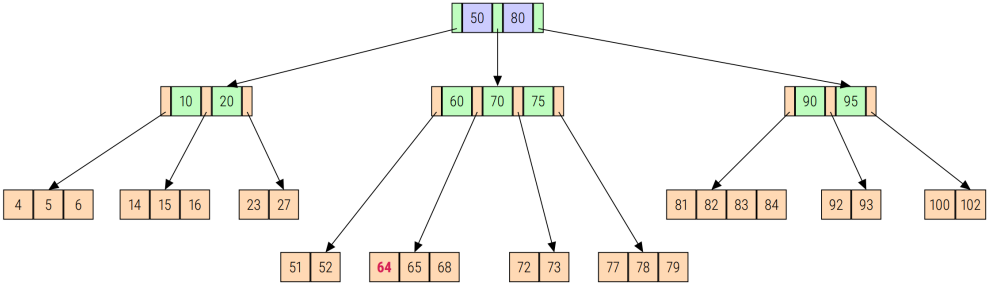
Cases on Deletion of B Tree



B-Tree Deletion: Case 1.(a) — Before

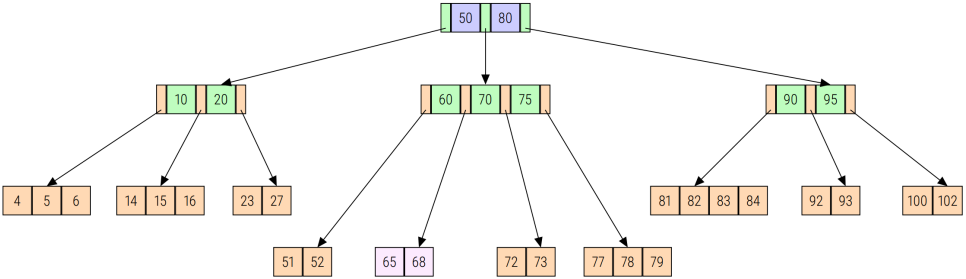
Case 1(a): Delete a key from a leaf node where the node has more than the minimum number of keys. *Just delete it since there are enough keys even after deleting this, so it incurs no extra process of adjustment (Here m=5 i.e. Max Children 5 and Max Keys 4 for all following examples)*

Delete key: **64**



B-Tree Deletion: Case 1.(a) — After

After deleting key **64**:



Same is the case for 16/77/84 and so on.

B-Tree Deletion: The 3-Step Decision

When deleting from a **leaf node with minimum keys**, follow this decision order:

Step 1: Check Left Sibling

- Does left sibling have **more than minimum** keys?
- **YES** $\longrightarrow$ **Borrow from Left** (Case 1b-i)
- Max value from left goes up to parent, parent's key comes down

Step 2: Check Right Sibling (if left cannot lend)

- Does right sibling have **more than minimum** keys?
- **YES** $\longrightarrow$ **Borrow from Right** (Case 1b-ii)
- Min value from right goes up to parent, parent's key comes down

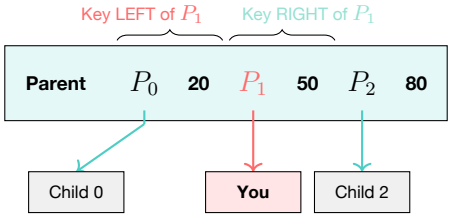
Step 3: Merge (if neither can lend)

- Both siblings at minimum $\longrightarrow$ **Cannot borrow**
- **MERGE** with left or right sibling + parent's separator key (Case 1b-iii)

Understanding the “Borrow” Mechanism

The Core Question: Which parent key comes down when borrowing?

Parent Structure: Pointers sit **between** the keys



If you are at P_1 (highlighted in red) and need to borrow:

- From **LEFT sibling** (Child 0): Key **20** comes down — it’s the key between P_0 and P_1
- From **RIGHT sibling** (Child 2): Key **50** comes down — it’s the key between P_1 and P_2

Understanding the “Borrow” Mechanism (cont.)

The Separator Rule

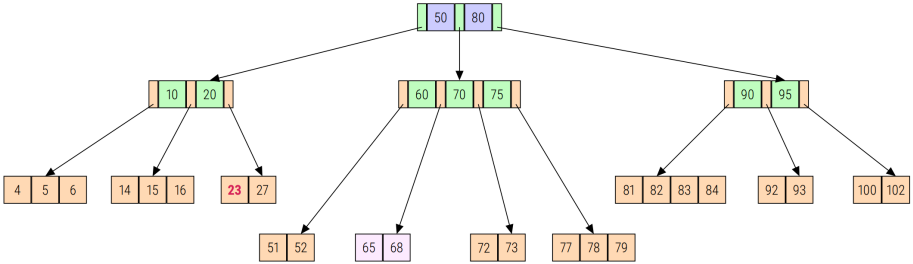
The parent key sitting between your pointer and your sibling’s pointer is the **bridge** — it comes down to you, while your sibling’s key goes up to replace it.

B-Tree Deletion: Case 1.(b)(i) — Before

Case 1(b)(i): Delete from leaf where node has minimum keys, but sibling has extra keys — **borrow from left sibling**.

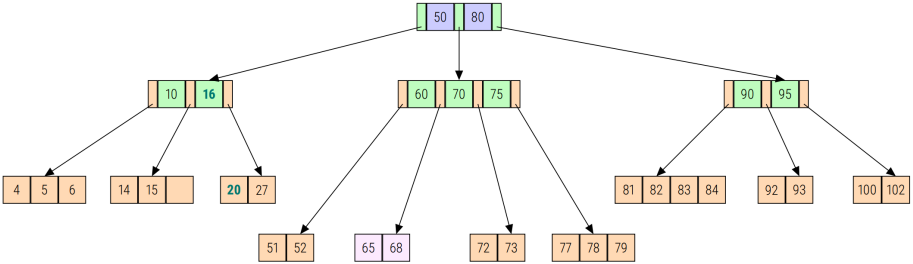
From Left Sibling max value will go up to its immediate root (iroot) and the root will be replaced by the key value (i.e. delete item). Here 16 will go up and send the iroot i.e. 20 (simply 20 will go down) in place of the delete item i.e. 23 (simply 23 will be removed).

Delete key: **23**



B-Tree Deletion: Case 1.(b)(i) — After

After deleting key **23** (borrowed from left sibling):

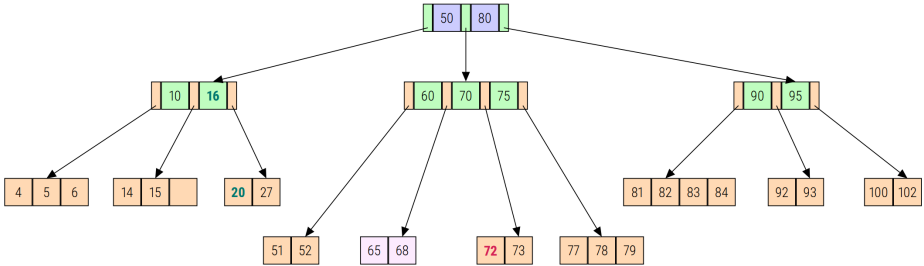


B-Tree Deletion: Case 1.(b)(ii) — Before

Case 1(b)(ii): Delete from leaf where node has minimum keys, but sibling has extra keys — **borrow from right sibling**.

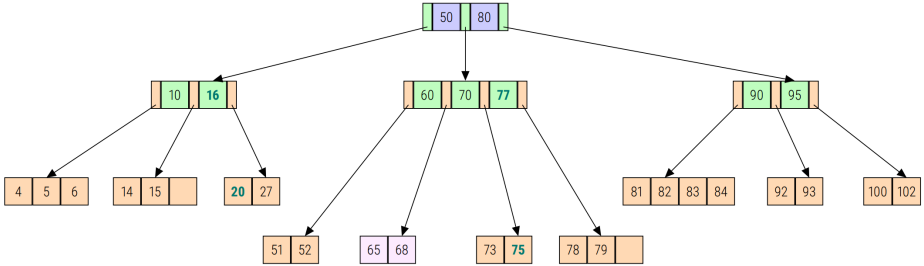
From Right Sibling min value will go up to its immediate root (iroot) and the root will be replaced by the key value (i.e. delete item). Here 77 will go up and send the iroot i.e. 75 (simply 75 will go down) in place of the delete item i.e. 72 (simply 72 will be removed).

Delete key: **72**



B-Tree Deletion: Case 1.(b)(ii) — After

After deleting key **72** (borrowed from right sibling):

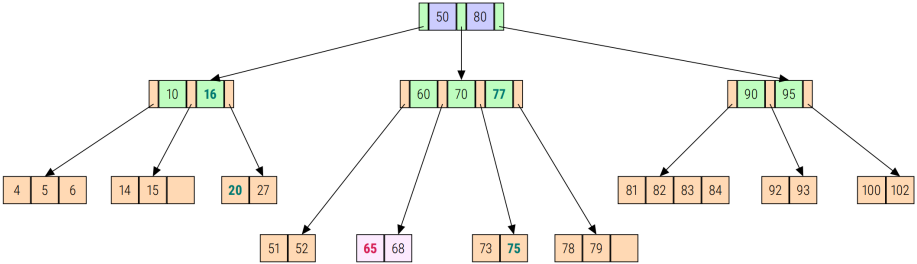


B-Tree Deletion: Case 1.(b)(iii) — Before

Case 1(b)(iii): Delete from leaf where node and siblings both have minimum keys — **merge with a sibling**.

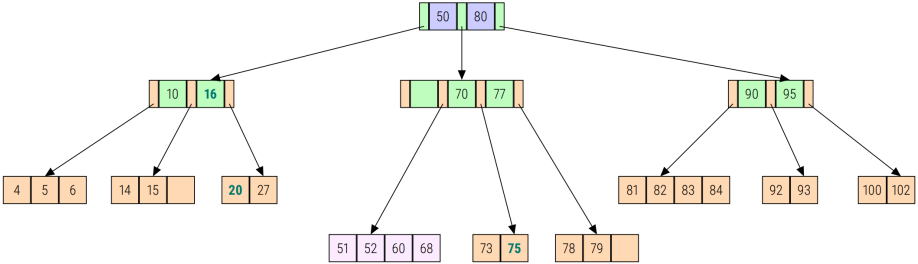
Either Left or Right merge with immediate parent node merging. (Left one is shown here) Both node (51,52) and node (-,68) will be merged with 60 as it is the immediate parent node.

Delete key: **65**



B-Tree Deletion: Case 1.(b)(iii) — After

After deleting key **65**:



B+ Tree

B-Tree to B+ Tree Conversion

1. **Data Storage: B-Tree:** Keys and records can be in **both** internal and leaf nodes. **B+ Tree:** All records are stored only in **leaf nodes**. Internal nodes store only keys as separators
2. **Key Duplication: B-Tree:** Each key appears only once. **B+ Tree:** Keys in internal nodes are **copies** of leaf keys (stored more than once)
3. **Leaf Node Links: B+ Tree only:** All leaf nodes are connected via a **linked list** (right pointer) This makes **range queries** very efficient

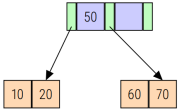


Figure: B Tree

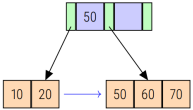


Figure: B+ Tree

Note

In B+ Tree, nodes at the right are **greater or equal** to its parent node

B+ Tree Insertion: Overview

The Insertion Process:

1. **Search** for the appropriate leaf node
   - Traverse from root using key comparisons
   - Always ends at a **leaf node** (all records are here!)
2. **Insert** into the leaf node
   - Keep keys in sorted order
   - If space available: **Insertion complete!**
3. If leaf is full $\rightarrow$ **SPLIT**
   - Sort all keys (including new one)
   - First half: stays in current leaf
   - Second half: moves to **new leaf**
   - **Middle key COPIES up** to parent (separator!)
4. If parent full $\rightarrow$ Split propagates upward
   - May create new root

Key Differences from B-Tree:

B-Tree Split

Middle key **MOVES UP** to parent.
That key is **GONE** from child.

B+ Tree Split

Middle key **COPIES UP** to parent.
Key **STAYS** in leaf (duplicates!).
Leaf nodes stay linked.

Critical Property:

All leaves linked via pointers:

$$[LeafA] \rightarrow [LeafB] \rightarrow [LeafC]$$

This enables fast **range queries**!

Interactive B+ Tree Visualization

Go to this link to simulate and try B+Tree:

`https://www.cs.usfca.edu/~galles/visualization/BPlusTree.html`

Tip

Use this visualization tool to practice insertion and deletion operations. Set different values of m and see how the tree structure changes!

How B+ Tree is Built from Scratch

The Core Difference: All records live ONLY in leaves. Internal nodes are just **road signs** (separators).

Step-by-Step Building Process:

1. **Insert into Leaf:** Always insert the new key into a **leaf node** (sorted order)
2. **Check Capacity:** If leaf has space, done! If **full**, split the leaf
3. **Split the Leaf:**
   - Sort all keys including the new one
   - First half stays in current leaf
   - Second half goes to a **brand new leaf**
   - **Middle key COPIES UP** to parent (stays in leaf too!), it is the first key of the right leaf.
4. **Link the Leaves:** Connect the two leaves with a **right pointer** (linked list!)
5. **Parent Full?** If parent has no space, **split parent** too (middle key copies up again)

How B+ Tree is Built from Scratch (cont.)

B-Tree Split

Middle key **MOVES UP**

Key **disappears** from child

B+ Tree Split

Middle key **COPIES UP**

Key **stays** in leaf (duplicate!)

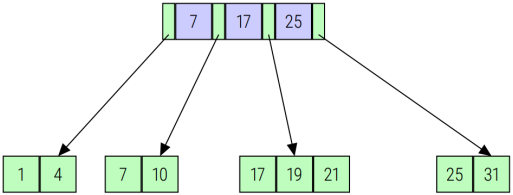
Why Linked Leaves Matter

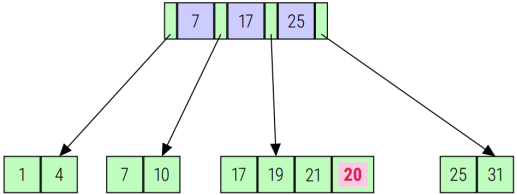
All leaf nodes connected in a chain: $[LeafA] \rightarrow [LeafB] \rightarrow [LeafC]$

This makes **range queries** incredibly fast — just walk the leaf chain!

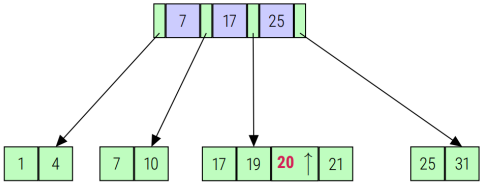
B+Tree Construction

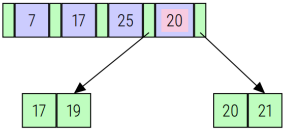
Dataset: **{1,4,7,10,17,21,31,25,19,20,28,42}**, m = 4 i.e. Max Child 4, Max Key 3 Insert key: **20**

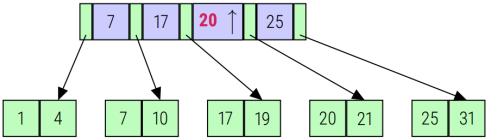




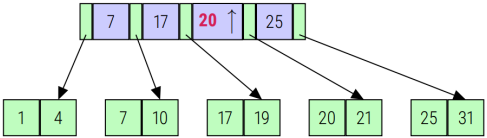
B+Tree Construction (cont.)

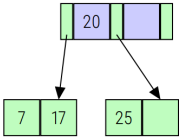






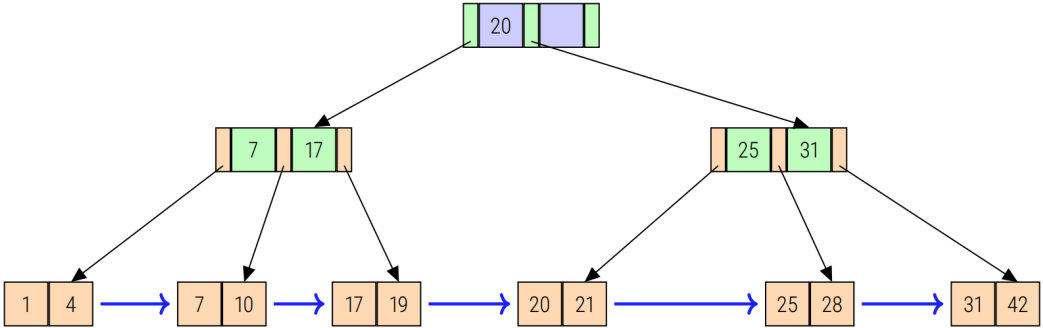
B+Tree Construction (cont.)





B+-Tree Construction: Final tree

Insert key: **28, 42** *(After Completing insertion of 20)*

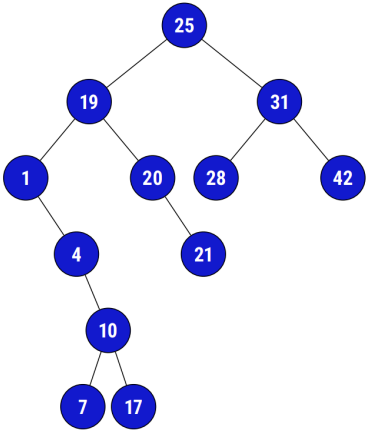


BST, B Tree and B+ Tree in Database Query

- **BST is not suitable** for handling large values which is common in databases
- **B Tree** is very fast for **single-point SQL queries**
- **B+ Tree** is very fast for **range queries**
- We will visualize this by drawing the BST and B Tree for the same dataset and m value

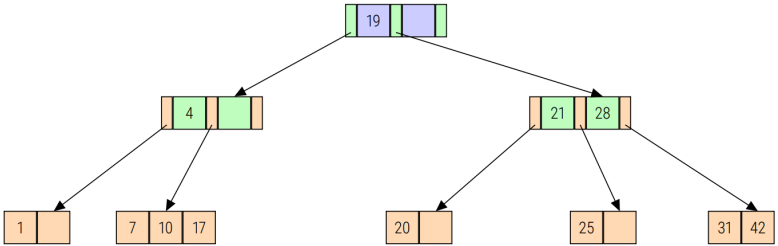
BST for the Same Dataset

Dataset: **25, 19, 20, 1, 4, 10, 17, 21, 31, 28, 42, 7**



B Tree for the Same Dataset with $m = 4$

Dataset: **25, 19, 20, 1, 4, 10, 17, 21, 31, 28, 42, 7**



Indexing and SQL

Dataset: **25, 19, 20, 1, 4, 10, 17, 21, 31, 28, 42, 7** (the order determines the skew of the tree)

Now consider the following SQL:

```
SELECT ADDRESS
FROM EMP
WHERE ID = 21;
```

Number of comparisons needed to execute the query:

1. In **BST** it is: **4**
2. In **B Tree** it is: **2**
3. In **B+ Tree** it is: **3**

Indexing and SQL: Range Query

Dataset: **25, 19, 20, 1, 4, 10, 17, 21, 31, 28, 42, 7**

Now consider the following SQL:

```sql
SELECT ADDRESS
FROM EMP
WHERE ID BETWEEN 7 AND 21;
```

The range covers values: **7, 10, 17, 19, 20, 21**. Number of comparisons needed:

1. In **BST** it is: $6 + 5 + 6 + 2 + 3 + 4 =$ **26**
2. In **B Tree** it is: $3 + 0 + 0 + 2 + 3 + 2 =$ **10**
3. In **B+ Tree** it is: **2 (only!)**

Key Takeaway

B+ Tree dominates for **range queries** because all leaf nodes are connected via a **linked list** — once the start is found, traverse the list directly.

Bitmap Indices

Bitmap Indices: Concept

Definition

A **bitmap** is simply an **array of bits** (0s and 1s). In its simplest form, a bitmap index on attribute A consists of one bitmap for **each distinct value** that A can take. Each bitmap has **as many bits as the number of records** in the relation.

Key Characteristics:

- Each bit position represents **one record** (1 = has value, 0 = doesn't have value)
- Best for attributes with **low cardinality** (few distinct values)
- Examples: `Gender` (M/F), `Blood Group` (A, B, AB, O), `Income Level` (L1, L2, L3)
- Classes must be **non-overlapping** (mutually exclusive)

Why Bitmap?

Multiple conditions can be combined using simple **bitwise operations** (AND, OR, NOT) — extremely fast on modern CPUs!

Bitmap Indices: Example

Consider a `citizens` table with 8 records and 4 income levels (L1, L2, L3, L4):

Rec#	Name	Gender	Income	Rec#	Name	Gender	Income
1	Alice	M	L1	5	Eve	M	L2
2	Bob	F	L2	6	Frank	M	L4
3	Carol	M	L1	7	Grace	F	L3
4	David	F	L1	8	Henry	M	L3

Bitmap Vectors for Gender (2 distinct values):

	1	2	3	4	5	6	7	8
Gender='M'	1	0	1	0	1	1	0	1
Gender='F'	0	1	0	1	0	0	1	0

Bitmap Vectors for Income (4 distinct values) — One vector per value:

	1	2	3	4	5	6	7	8
Income='L1'	1	0	1	1	0	0	0	0
Income='L2'	0	1	0	0	1	0	0	0
Income='L3'	0	0	0	0	0	0	1	1
Income='L4'	0	0	0	0	0	1	0	0

Bitmap Indices: Example (cont.)

Key Insight

Each distinct value gets its **own bitmap vector**.

For Income with 4 levels: L1, L2, L3, L4 $\longrightarrow$ **4 separate bit vectors**!

Bitmap Indices: AND Operation

Query: `SELECT ID FROM citizens WHERE gender='M' AND income='L1';`

Step 1: Retrieve the relevant bitmap vectors

	1	**2**	**3**	**4**	**5**	**6**	**7**	**8**
Gender='M'	1	0	1	0	1	1	0	1
Income='L1'	1	0	1	1	0	0	0	0

Step 2: Perform Bitwise AND (1 AND 1 = 1, else 0)

	1	**2**	**3**	**4**	**5**	**6**	**7**	**8**	
Gender='M'	1	0	1	0	1	1	0	1	
Income='L1'	1	0	1	1	0	0	0	0	
AND Result	**1**	**0**	**1**	**0**	**0**	**0**	**0**	**0**	$\rightarrow$ Recs **1, 3**

Bitmap Indices: AND Operation (cont.)

Result

Records 1, 3 match: Alice (M, L1) and Carol (M, L1)

No Record Access Needed!

We found the answer using only bit vectors — **no disk I/O** to fetch actual records!

Bitmap Indices: OR Operation Example

Query: `SELECT ID FROM citizens WHERE income='L1' OR income='L3';`

Step 1: Retrieve bitmaps for both income levels

	1	**2**	**3**	**4**	**5**	**6**	**7**	**8**
Income='L1'	1	0	1	1	0	0	0	0
Income='L3'	0	0	0	0	0	0	1	1

Step 2: Perform Bitwise OR (0 OR 0 = 0, else 1)

	1	**2**	**3**	**4**	**5**	**6**	**7**	**8**	
Income='L1'	1	0	1	1	0	0	0	0	
Income='L3'	0	0	0	0	0	0	1	1	
OR Result	**1**	**0**	**1**	**1**	**0**	**0**	**1**	**1**	$\rightarrow$ Recs **1,3,4,7,8**

Bitmap Indices: OR Operation Example (cont.)

Result

Records 1, 3, 4, 7, 8 have either L1 or L3 income

Multiple Values = Multiple Bitmaps

For OR conditions across the same attribute, we simply OR the corresponding bitmaps!

Bitmap Indices: COUNT(*)

Query: `SELECT COUNT(*) FROM citizens WHERE gender='M' AND income='L1';`

Traditional Approach:

1. Find all records with gender='M' $\longrightarrow$ **Disk I/O**
2. Check which have income='L1' $\longrightarrow$ **More Disk I/O**
3. Count the matching records $\longrightarrow$ **Process records**

With Bitmap Index (No Record Access!):

	1	**2**	**3**	**4**	**5**	**6**	**7**	**8**
Gender='M'	1	0	1	0	1	1	0	1
Income='L1'	1	0	1	1	0	0	0	0
AND Result	1	0	1	0	0	0	0	0

Bitmap Indices: COUNT(*) (cont.)

Just Count the 1s!

AND result has **two 1s** at positions 1 and 3 $\longrightarrow$ **Answer: 2**

The Special Benefit

For `COUNT(*)`, **no need to look at actual records at all**!

Just AND the vectors and count bits — all in **memory**, no disk access!

Bitmap Indices: Summary

Best For:

- Low cardinality attributes
- Gender, Blood Group, Status flags
- Data warehouse queries
- Multiple AND/OR conditions

Advantages:

- Very fast bitwise operations
- Efficient storage for low-cardinality
- COUNT(*) without record access
- Easy parallel processing

Not Suitable For:

- High cardinality (many unique values)
- Names, addresses, timestamps
- Range queries (without compression)

Limitations:

- Space grows with: values $\times$ records
- Updates are expensive
- Not for OLTP (many writes)

Questions?